

Monte Carlo for delta hedging of AAO. Code explanation

Zakhar Nefedov

Contents

1	Notation and variables	2
2	Notes	2
3	General class structure	3
4	Inputs and initialisation	4
5	Hedging block	5
5.1	Conditional Brownian motion	5
5.2	Delta hedge	5
6	Geometric asian options	7
6.1	Closed form	7
6.1.1	Price	7
6.1.2	Delta	8
6.2	Path-dependent	9
6.2.1	Price	9
6.2.2	Delta	10
7	Arithmetic asian options	10
7.1	Price	10
7.2	Delta	11
8	Variance reduction	12
8.1	Antithetic variates	12
8.2	Kemna-Vorst method	12
A	Appendix	14
A.1	Proof of 6.1.1	14
A.2	Linear approximation of Brownian motion (6.2.1)	15
A.3	Proof of 6.2.2	15

1 Notation and variables

- i GAO - Geometric Asian Option
- ii AAO - Arithmetic Asian Option
- iii $\tilde{\mathbb{E}}, \tilde{\mathbb{P}}$ - expectation, probability under standard risk-neutral probability measure
- iv $\widetilde{W}(t)$ - stochastic process that is a Brownian motion under $\tilde{\mathbb{P}}$. I.e $d\widetilde{W}(t)d\widetilde{W}(t) = t$, $\tilde{\mathbb{E}}\widetilde{W}(t) = 0$ and $\text{Var}(\widetilde{W}(t)) = t$
- v $\mathbb{I}_{\{A\}}$ - indicator function for condition A . I.e $\mathbb{I}_{\{A\}} = 1$ if A is true and 0 otherwise.
- vi $A^{(G)}(t)$ - price of the geometric asian option at time t
- vii $\Delta^{(G)}(t)$ - delta of the geometric asian option at time t
- viii $A^{(A)}(t)$ - price of the arithmetic asian option at time t
- ix $\Delta^{(A)}(t)$ - delta of the arithmetic asian option at time t
- x T - time of expiration ($1 = 1$ year)
- xi R - risk-free interest rate
- xii σ - implied volatility of stock
- xiii K - strike price
- xiv $S(t)$ - stock price at time t

2 Notes

- i Implied volatility σ and interest rate R are assumed to be constant. Dividend payment is assumed to be zero.
- ii Brownian motion is modeled using n discrete increments, equal to random sample based of normal random variable with mean 0, s.d. $\frac{T}{n}$. (antithetic variates are used, see [8.1](#))
- iii At time t , price and delta are equal to expectation conditional on filtration $\mathcal{F}(t)$, i.e. $A^{(A)}(t) = \dots \tilde{\mathbb{E}}[\dots | \mathcal{F}(t)]$. However, all the formulas given later are for $t = 0$. This is because underlying Brownian motion is changed to include filtration, and hence, formulas remain the same for every time increment
- iv Stock price as function of time is taken as the standard geometric Brownian motion $S(t) = S(0) \exp \left\{ \left(R - \frac{1}{2}\sigma \right) t + \sigma \widetilde{W}(t) \right\}$ which satisfies the differential equation $dS(t) = \sigma S(t) d\widetilde{W}(t) + RS(t)dt$.
- v All the calculations in the code are vectorised using **Eigen**, but formulas are given as for single simulated path of Brownian motion.

3 General class structure

The outline of the code is given below. For function type `function[04](01-03){description}`, [03] is the internal order number of the function used only in this section. Parameters ([...],...) represent the direct dependencies of the function (i.e. in the example above, function [04] requires [01], [02] and [03] to run first, but if [03] has a dependency on [05], it is not written in dependencies of [04]).

(c.f. - complementary function)

```
class mcs_kvcv_aao{
public:
    double norm_cdf[00]() {Normal cumulative distribution function}
    mcs_kvcv_aao()
        {class constructor, see 4.}
    double sim_run[01](03,16,17)
        {main function, performs the hedge and
         returns the final portfolio value, see 5.2}
private:
    ... variables definition
    Eigen::RowVectorXd brownian_cond_conv[02]() {Used to perform filtration, see 5.1}
    Eigen::MatrixXd brownian_cond[03](02) {Used to perform filtration, see 5.1}

    double cfs_gao_prc[04](00) {Closed form solution for GAO price, see 6.1.1}
    double cfs_gao_prc[05](00) {Closed form solution for GAO delta, see 6.1.2}

    Eigen::MatrixXd step_t[06]() {c.f., see 6.2.1}
    Eigen::MatrixXd gao_beta_t[07](06) {c.f., see 6.2.1}
    Eigen::VectorXd gao_s_bar[08](07) {c.f., see 6.2.1}
    Eigen::VectorXd pd_gao_prc[09](08) {Path dependent GAO price, see 6.2.1}

    Eigen::VectorXd pd_gao_dlt[10](08) {Path dependent GAO delta, see 6.2.2}

    Eigen::MatrixXd delta_t[11]() {c.f., see 7.1}
    Eigen::MatrixXd aao_alpha_t[12](11) {c.f., see 7.1}
    Eigen::MatrixXd aao_beta_t[13](11) {c.f., see 7.1}
    Eigen::MatrixXd aao_s_bar[14](12,13) {c.f., see 7.1}
    Eigen::VectorXd pd_aao_prc[15](14) {Path dependent AAO price, see 7.1}

    Eigen::VectorXd pd_aao_dlt[16](14) {Path dependent AAO delta, see 7.2}

    double kemna_vorst_block_prc[16](04,09,15)
        {Kemna-Vorst geometric control variate for price, see 8.2}
    double kemna_vorst_block_prc[17](05,10,16)
        {Kemna-Vorst geometric control variate for delta, see 8.2}
};
```

4 Inputs and initialisation

Simulation takes the following parameters in order:

- i T - double; as in 1, viii
- ii n - int; number of discrete steps in the simulated Brownian motion.
- iii M - int; number of simulation runs per rebalancing step.
- iv $R, \sigma, K, S(0)$ - double; parameters as in 1, ix - xii
- v $\mathbf{S}_T$ - Eigen::RowVectorXd; vector of real stock prices from $t = 0$ to $t = T$. Hence, as is, simulation only works on past data, which was done for testing. But, can easily be adapted to take incoming stock price data
- vi s - unsigned long long; seed used for generation of Brownian motion paths. (Not necessary)

```
mcs_kvqv_aao(double in_ft, int in_bs, int in_mcr, double in_ir, double in_v,
             double in_sp, double in_ip, Eigen::RowVectorXd in_stockprc,
             unsigned long long seed = std::random_device{}()):
gen(seed), n_brownian_stp(in_bs), n_mcs_run(in_mcr), p_final_t(in_ft),
p_int_rt(in_ir), p_vol(in_v), p_strk_prc(in_sp), p_init_prc(in_ip),
v_stock_prcs(in_stockprc)
{
    double _sd = std::sqrt(p_final_t / n_brownian_stp);
    Eigen::MatrixXd m_brownian_nz = Eigen::MatrixXd::Zero(n_mcs_run *
        0.5, n_brownian_stp).unaryExpr([this, _sd](double)
        {return(gaussian(gen) * _sd);});
    for(int i = 1; i < m_brownian_nz.cols(); i++)
        {m_brownian_nz.col(i) += m_brownian_nz.col(i-1);}
    Eigen::MatrixXd m_brownian_az(m_brownian_nz.rows(),
        m_brownian_nz.cols() + 1);
    m_brownian_az << Eigen::MatrixXd::Zero(n_mcs_run * 0.5, 1), m_brownian_nz;
    Eigen::MatrixXd m_brownian_l(m_brownian_az.rows() * 2, m_brownian_az.cols());
    m_brownian_l << m_brownian_az, (-1) * m_brownian_az;
    m_brownian = m_brownian_l;
    v_timesteps = Eigen::VectorXd::LinSpaced(n_brownian_stp + 1, 0, p_final_t);

    gao_alpha_t = std::exp(0.5 * p_final_t *
        (p_int_rt - 0.5 * std::pow(p_vol, 2)));
    gamma_t = std::exp(-0.5 * p_int_rt * p_final_t -
        (std::pow(p_vol, 2) * p_final_t) / 12);
    lambda_t = (p_final_t / p_vol) * std::log(p_strk_prc / p_init_prc) -
        (std::pow(p_final_t, 2) / (2 * p_vol)) *
        (p_int_rt - 0.5 * std::pow(p_vol, 2));
}
```

Here, `v_timesteps` is a column vector of time increments, `m_brownian` is a $M \times n$ matrix, with each row representing separate simulated run, each column representing discrete step (see 2, ii and 8.1 for more information)

5 Hedging block

5.1 Conditional Brownian motion

- i Function (private) `brownian_cond_conv`. Type `Eigen::MatrixXd`. Inputs: `int _in_n` - index corresponding to the time step, based on which filtration is performed.
Converts stock prices known up to time t to their underlying Brownian motion as below:

$$W(t) = \frac{1}{\sigma} \log \left(\frac{S(t)}{S(0)} \right) - \frac{t}{\sigma} \left(R - \frac{1}{2} \sigma^2 \right)$$

```
Eigen::RowVectorXd brownian_cond_conv(int _in_n)
{
    Eigen::RowVectorXd _stock_cond = v_stock_prcs.leftCols(_in_n);
    Eigen::RowVectorXd _brownian_cond = (_stock_cond / p_init_prc).array().log() /
        p_vol - v_timesteps.head(_in_n).transpose() / p_vol *
        (p_int_rt - 0.5 * std::pow(p_vol, 2));
    return(_brownian_cond);
}
```

- ii Function (private) `brownian_cond`. Type `Eigen::MatrixXd`. Inputs: `int _in_n` - index corresponding to the time step, based on which filtration is performed.
Replaces first `in_n` steps of each simulated Brownian motion path with real stock price data. This has the identical effect to taking conditional expectation (see 2, iii)

```
Eigen::MatrixXd brownian_cond(int _in_n)
{
    Eigen::RowVectorXd _v_brownian_cond = brownian_cond_conv(_in_n);
    Eigen::MatrixXd _m_brownian_end =
        m_brownian.leftCols(n_brownian_stp - _in_n + 1).array() +
        _v_brownian_cond(Eigen::last);
    Eigen::MatrixXd _m_brownian_beg = _v_brownian_cond.replicate(n_mcs_run, 1);
    Eigen::MatrixXd _brownian_cond(_m_brownian_beg.rows(),
        _m_brownian_beg.cols() + _m_brownian_end.cols());
    _brownian_cond << _m_brownian_beg, _m_brownian_end;
    return(_brownian_cond);
}
```

5.2 Delta hedge

Function (**public**) `sim_run`. Type `double`. Inputs: None
Performs the delta hedge, returns the final portfolio price. Calculates each time increment as follows:

$$A_{KV}^{(A)}(t) = \Delta_{KV}^{(A)}(t)S(t) + B(t)$$

Where $A_{KV}^{(A)}$ and $\Delta_{KV}^{(A)}$ are price and delta of arithmetic Asian option received through Kemna-Vorst geometric control variates method (See 8.2). Here $\exp\{\frac{TR}{n}\}$ of interest rate is accumulated on the cash balance $B(t)$ per time step:

```

double sim_run()
{
    double _init_cap = kemna_vorst_block_prc(m_brownian);
    double _prev_dlt = (n_brownian_stp / (n_brownian_stp + 1)) *
        kemna_vorst_block_dlt(m_brownian);
    double _debt_ira = _init_cap - p_init_prc * _prev_dlt;
    for(int i = 2; i <= n_brownian_stp; i++)
    {
        Eigen::MatrixXd _m_new_brownian = brownian_cond(i);
        _debt_ira = _debt_ira * std::exp(p_int_rt * p_final_t / n_brownian_stp);
        double _new_dlt = kemna_vorst_block_dlt(_m_new_brownian);
        double _scaled = ((n_brownian_stp - i + 1)*1.0) / ((n_brownian_stp + 1)*1.0);
        _debt_ira = _debt_ira - v_stock_prcs(i-1) * (_scaled * _new_dlt - _prev_dlt);
        _prev_dlt = _scaled * _new_dlt;
    }
    return(_prev_dlt / (n_brownian_stp + 1) * v_stock_prcs(Eigen::last) + _debt_ira);
}

```

6 Geometric asian options

6.1 Closed form

6.1.1 Price

Geometric mean over continuous function $f(x)$ on the range $(0, T)$ is:

$$\Gamma_{f(x)} = \exp \left\{ \frac{1}{T} \int_0^T \ln S(t) dt \right\}$$

$$A^G(0) = e^{-RT} \tilde{\mathbb{E}} \left[\left(\exp \left\{ \frac{1}{T} \int_0^T \ln S(t) dt \right\} - K \right)^+ \right]$$

Solving for expectation, value of the Geometric Asian Option is:

$$A^G(0) = S(0) \gamma_T \left(1 - N \left(\lambda_T, \frac{\sigma T^2}{3}, \sqrt{\frac{T^3}{3}} \right) \right) - K e^{-RT} \left(1 - N \left(\lambda_T, 0, \sqrt{\frac{T^3}{3}} \right) \right)$$

```
double cfs_gao_prc()
{
    double _sd = std::sqrt(std::pow(p_final_t, 3) / 3);
    double _mean = p_vol * std::pow(p_final_t, 2) / 3;
    double _cfs_gao_prc = p_init_prc *
        gamma_t * (1 - norm_cdf(lambda_t, _mean, _sd)) -
        p_strk_prc * std::exp(-p_int_rt * p_final_t) *
        (1 - norm_cdf(lambda_t, 0, _sd));
    return(_cfs_gao_prc);
}
```

where $N(x, \mu, \sigma)$ is a cumulative density function for normal RV from $-\infty$ to x , with mean μ and standard deviation σ .

```
double norm_cdf(double x, double mu, double sigma)
{return(0.5 * (1 + std::erf((x - mu) / (std::sqrt(2) * sigma))));}
```

The functions γ_T and λ_T are defined in the constructor (see 4), but are repeated here:

$$\gamma_T = \exp \left\{ -\frac{1}{2} RT - \frac{1}{12} \sigma^2 T \right\}$$

```
gamma_t = std::exp(-0.5 * p_int_rt * p_final_t - (std::pow(p_vol, 2) * p_final_t) / 12);
```

$$\lambda_T = \frac{T}{\sigma} \ln \left(\frac{K}{S(0)} \right) - \frac{T^2}{2\sigma} \left(R - \frac{1}{2} \sigma^2 \right)$$

```
lambda_t = (p_final_t / p_vol) * std::log(p_strk_prc / p_init_prc) -
    (std::pow(p_final_t, 2) / (2 * p_vol)) * (p_int_rt - 0.5 * std::pow(p_vol, 2));
```

6.1.2 Delta

$$\begin{aligned}
\Delta^{(G)}(0) &= \frac{\partial A^{(G)}(0)}{\partial S(0)} \\
&= \gamma_T \left(1 - N \left(\lambda_T, \frac{\sigma T^2}{3}, \sqrt{\frac{T^3}{3}} \right) \right) + \frac{\gamma_T}{\sigma} \sqrt{\frac{3}{2\pi T}} \exp \left\{ -\frac{3}{2T^3} \left(\lambda_T^2 - \frac{2}{3} \lambda_T \sigma T^2 + \frac{\sigma^2 T^4}{9} \right) \right\} \\
&\quad - \frac{K e^{-RT}}{\sigma S(0)} \sqrt{\frac{3}{2\pi T}} \exp \left\{ -\frac{3\lambda_T^2}{2T^3} \right\} \\
&= a + b - c
\end{aligned}$$

where:

$$\begin{aligned}
\phi &= \frac{1}{\sigma} \sqrt{\frac{3}{2\pi T}} \\
a &= \gamma_T \left(1 - N \left(\lambda_T, \frac{\sigma T^2}{3}, \sqrt{\frac{T^3}{3}} \right) \right) \\
b &= \gamma_T \phi \exp \left\{ -\frac{3}{2T^3} \left(\lambda_T^2 - \frac{2}{3} \lambda_T \sigma T^2 + \frac{\sigma^2 T^4}{9} \right) \right\} \\
c &= \frac{K e^{-RT}}{S(0)} \phi \exp \left\{ -\frac{3\lambda_T^2}{2T^3} \right\}
\end{aligned}$$

```

double cfs_gao_dlt()
{
    double _sd = std::sqrt(std::pow(p_final_t, 3) / 3);
    double _mean = p_vol * std::pow(p_final_t, 2) / 3;
    double _term_phi = (1 / p_vol) * std::sqrt(3 / (2 * p_final_t * std::numbers::pi));
    double _term_a = gamma_t * (1 - norm_cdf(lambda_t, _mean, _sd));
    double _term_b = gamma_t * _term_phi * std::exp(-3 / (2 * std::pow(p_final_t, 3)) *
        (std::pow(lambda_t, 2) - (2.0 / 3.0) * lambda_t * p_vol *
        std::pow(p_final_t, 2) + std::pow(p_vol, 2) * std::pow(p_final_t, 4) / 9));
    double _term_c = (_term_phi * p_strk_prc * std::exp(-p_int_rt * p_final_t) /
        p_init_prc) * std::exp(-3 * std::pow(lambda_t, 2) /
        (2 * std::pow(p_final_t, 3)));
    return(_term_a + _term_b - _term_c);
}

```

where γ_T and λ_T are as in [6.1.1](#).

6.2 Path-dependent

6.2.1 Price

The price of a GAO at time 0 is defined as:

$$A^G(0) = e^{-RT} \tilde{\mathbb{E}} \left[\left(\exp \left\{ \frac{1}{T} \int_0^T \ln S(t) dt \right\} - K \right)^+ \right]$$

Expanding:

$$\begin{aligned} A^G(0) &= e^{-RT} \tilde{\mathbb{E}} \left[\left(S(0) \exp \left\{ \frac{1}{2} \left(R - \frac{1}{2} \sigma^2 \right) T + \frac{\sigma}{2} \sum_{i=0}^{i_T} \frac{w_{t_{i+1}} + w_{t_i}}{n} \right\} - K \right)^+ \right] \\ &= e^{-RT} \tilde{\mathbb{E}} \left[(S(0) \alpha_T \beta_T - K)^+ \right] = e^{-RT} \tilde{\mathbb{E}} \left[(\bar{S}(T) - K)^+ \right] \end{aligned}$$

Hence, under filtration $\mathcal{F}(T)$, this simplifies to:

$$A^{(G)}(0) = e^{-RT} (\bar{S}(T) - K)^+$$

```
Eigen::VectorXd pd_gao_prc(const Eigen::MatrixXd _in_m_brownian)
{return(std::exp(-p_int_rt * p_final_t) *
      (gao_s_bar(_in_m_brownian).array() - p_strk_prc).max(0.0));}
```

where:

$$\bar{S}(T) = S(0) \alpha_T \beta_T$$

```
Eigen::VectorXd gao_s_bar(const Eigen::MatrixXd _in_m_brownian)
{return(p_init_prc * gao_alpha_t * gao_beta_t(_in_m_brownian));}
```

(In the program, α_T is defined in the constructor, see [4](#), but the code is repeated in this section)

$$\alpha_T = \exp \left\{ \frac{1}{2} \left(R - \frac{1}{2} \sigma^2 \right) T \right\}$$

```
gao_alpha_t = std::exp(0.5 * p_final_t * (p_int_rt - 0.5 * std::pow(p_vol, 2)));
```

$$\beta_T = \exp \left\{ \frac{\sigma}{2T} \sum_{i=0}^{i_T} \left(\frac{T}{n} (w_{t_{i+1}} + w_{t_i}) \right) \right\} = \exp \left\{ \frac{\sigma}{2T} \sum_{i=0}^{i_T} s_t \right\}$$

```
Eigen::MatrixXd gao_beta_t(const Eigen::MatrixXd _in_m_brownian)
{return((step_t(_in_m_brownian).rowwise().sum() * p_vol /
      (2 * p_final_t)).array().exp());}
```

$$s_t = \frac{T}{n} (w_{t_{i+1}} + w_{t_i})$$

```
Eigen::MatrixXd step_t(const Eigen::MatrixXd _in_m_brownian)
{
    Eigen::MatrixXd w_a = _in_m_brownian.leftCols(_in_m_brownian.cols() - 1);
    Eigen::MatrixXd w_b = _in_m_brownian.rightCols(_in_m_brownian.cols() - 1);
    return((p_final_t / n_brownian_stp) * (w_a + w_b));
}
```

6.2.2 Delta

Claim that:

$$\frac{\partial \bar{S}(T)}{\partial S(0)} = \frac{\bar{S}(T)}{S(0)}$$

(proof is provided in the appendix A.3). Hence,

$$\Delta^{(G)}(0) = \frac{\partial}{\partial S(0)} e^{-RT} \tilde{\mathbb{E}}[(\bar{S}(T) - K)^+] = e^{-RT} \tilde{\mathbb{E}}\left[\frac{\bar{S}(T)}{S(0)} \mathbb{I}_{\{\bar{S}(T) > K\}}\right]$$

```
Eigen::VectorXd pd_gao_dlt(const Eigen::MatrixXd _in_m_brownian)
{
    Eigen::VectorXd _s_bar = gao_s_bar(_in_m_brownian);
    Eigen::VectorXd _pd_gao_dlt = std::exp(-p_final_t * p_int_rt) *
        (_s_bar.array() / p_init_prc) * (_s_bar.array() - p_strk_prc > 0).cast<double>();
    return(_pd_gao_dlt);
}
```

Where $\bar{S}(T)$ is as in 6.2.1.

7 Arithmetic asian options

7.1 Price

The price of a AAO at time 0 is defined as:

$$A^{(A)}(0) = e^{-RT} \tilde{\mathbb{E}}\left[\left(\frac{1}{T} \int_0^T S(t) dt - K\right)^+\right] = e^{-RT} \tilde{\mathbb{E}}[(\bar{S}(T) - K)^+]$$

```
Eigen::VectorXd pd_aao_prc(const Eigen::MatrixXd _in_m_brownian)
{
    return(std::exp(-p_int_rt * p_final_t) *
        (aao_s_bar(_in_m_brownian).array() - p_strk_prc).max(0.0));
}
```

$$\bar{S}(T) = \frac{S(0)}{T} \sum_{i=0}^{i_T} \left(\frac{1}{\alpha_i} e^{\beta_i} (e^{\alpha_i t_{i+1}} - e^{\alpha_i t_i}) \right)$$

```
Eigen::MatrixXd aao_s_bar(const Eigen::MatrixXd _in_m_brownian)
{
    Eigen::MatrixXd alpha = aao_alpha_t(_in_m_brownian);
    Eigen::MatrixXd beta = aao_beta_t(_in_m_brownian);
    Eigen::MatrixXd step = alpha.array().inverse() * beta.array().exp() *
        ((alpha * v_timesteps.bottomRows(v_timesteps.rows() - 1).asDiagonal()).array().exp() -
        (alpha * v_timesteps.topRows(v_timesteps.rows() - 1).asDiagonal()).array().exp());
    return(step.rowwise().sum() * p_init_prc / p_final_t);
}
```

$$\alpha_i = \left(R - \frac{1}{2} \sigma^2 + \delta \cdot \sigma \right)$$

```

Eigen::MatrixXd aao_alpha_t(const Eigen::MatrixXd _in_m_brownian)
{
    return((p_int_rt - 0.5 * std::pow(p_vol, 2) +
        delta_t(_in_m_brownian).array() * p_vol).matrix());}

```

$$\beta_i = -\sigma(\delta \cdot t_i - w(t_i))$$

```

Eigen::MatrixXd aao_beta_t(Eigen::MatrixXd _in_m_brownian)
{
    Eigen::MatrixXd _w_a = _in_m_brownian.leftCols(_in_m_brownian.cols() - 1);
    Eigen::VectorXd _t_a = v_timesteps.topRows(v_timesteps.rows() - 1);
    Eigen::MatrixXd _delta = delta_t(_in_m_brownian);
    return(-p_vol * (_delta * _t_a.asDiagonal() - _w_a));
}

```

$$\delta_i = \frac{w(t_{(i+1)}) - w(t_i)}{t_{(i+1)} - t_i}$$

```

Eigen::MatrixXd delta_t(const Eigen::MatrixXd _in_m_brownian)
{
    Eigen::MatrixXd w_a = _in_m_brownian.leftCols(_in_m_brownian.cols() - 1);
    Eigen::MatrixXd w_b = _in_m_brownian.rightCols(_in_m_brownian.cols() - 1);
    return((w_b - w_a) / (p_final_t / n_brownian_stp));
}

```

7.2 Delta

Delta of AAO is defined as:

$$\Delta(0) = \frac{\partial}{\partial S(0)} e^{-RT} \tilde{\mathbb{E}} \left[(\bar{S}(T) - K)^+ \right] = e^{-RT} \tilde{\mathbb{E}} \left[\frac{\partial \bar{S}(T)}{\partial S(0)} \mathbb{I}_{\{\bar{S}(T) > K\}} \right] = e^{-RT} \tilde{\mathbb{E}} \left[\frac{\bar{S}(T)}{S(0)} \mathbb{I}_{\{\bar{S}(T) > K\}} \right]$$

Hence, under filtration $\mathcal{F}(T)$:

$$\Delta(0) = e^{-RT} \frac{\bar{S}(T)}{S(0)} \mathbb{I}_{\{\bar{S}(T) > K\}}$$

```

Eigen::VectorXd pd_aao_dlt(const Eigen::MatrixXd _in_m_brownian)
{
    Eigen::VectorXd _s_bar = aao_s_bar(_in_m_brownian);
    Eigen::VectorXd _pd_aao_dlt = std::exp(-p_final_t * p_int_rt) *
        (_s_bar.array() / p_init_prc) *
        (_s_bar.array() - p_strk_prc > 0).cast<double>();
    return(_pd_aao_dlt);
}

```

Where $\bar{S}(T)$ is as in [7.1](#).

8 Variance reduction

8.1 Antithetic variates

For n simulated Brownian motions, $\frac{n}{2}$ are simulated randomly:

$$W_1 = \begin{pmatrix} w_{11} & \cdots & w_{1T} \\ \vdots & \ddots & \vdots \\ w_{\frac{n}{2}1} & \cdots & w_{\frac{n}{2}T} \end{pmatrix}$$

and the other $\frac{n}{2}$ runs are taken as $W_2 = -W_1$. W_1 and W_2 are perfectly negatively correlated, reducing variance for Monte Carlo simulations run.

```
double _sd = std::sqrt(p_final_t / n_brownian_stp);
Eigen::MatrixXd m_brownian_nz = Eigen::MatrixXd::Zero(
    n_mcs_run * 0.5, n_brownian_stp).unaryExpr(
    [this, _sd](double){return(gaussian(gen) * _sd);});
for(int i = 1; i < m_brownian_nz.cols(); i++)
    {m_brownian_nz.col(i) += m_brownian_nz.col(i-1);}
Eigen::MatrixXd m_brownian_az(m_brownian_nz.rows(), m_brownian_nz.cols() + 1);
m_brownian_az << Eigen::MatrixXd::Zero(n_mcs_run * 0.5, 1), m_brownian_nz;
Eigen::MatrixXd m_brownian_l(m_brownian_az.rows() * 2, m_brownian_az.cols());
m_brownian_l << m_brownian_az, (-1) * m_brownian_az;
m_brownian = m_brownian_l;
```

8.2 Kemna-Vorst method

This is similar for both price and delta.

As arithmetic and geometric mean are correlated, can utilise the fact that there is closed-form solution for price and delta of geometric asian options.

$$A_{KV}^{(A)} = A^{(A)} - c(A^{(G)} - \tilde{\mathbb{E}}A^{(G)})$$

where:

$$c = \frac{\text{Cov}(A^{(A)}, A^{(G)})}{\text{Var}(A^{(G)})}$$

```
double kemna_vorst_block_prc(const Eigen::MatrixXd _in_m_brownian)
{
    Eigen::VectorXd _v_pd_gao_prc = pd_gao_prc(_in_m_brownian);
    Eigen::VectorXd _v_pd_aao_prc = pd_aao_prc(_in_m_brownian);
    double _cfs_gao_prc_mean = cfs_gao_prc();
    double _pd_gao_prc_mean = _v_pd_gao_prc.mean();
    double _pd_aao_prc_mean = _v_pd_aao_prc.mean();
    double _var_gao_prc = (_v_pd_gao_prc - _pd_gao_prc_mean).squaredNorm();
    double _c = _var_gao_prc > 0 ? (_v_pd_aao_prc - _pd_aao_prc_mean).dot(
        _v_pd_gao_prc - _pd_gao_prc_mean) / _var_gao_prc : 0;
    return(_pd_aao_prc_mean - _c * (_pd_gao_prc_mean - _cfs_gao_prc_mean));
}
```

Identically for delta:

$$\Delta_{KV}^{(A)} = \Delta^{(A)} - c(\Delta^{(G)} - \tilde{\mathbb{E}}\Delta^{(G)})$$

where:

$$c = \frac{\text{Cov}(\Delta^{(A)}, \Delta^{(G)})}{\text{Var}(\Delta^{(G)})}$$

```
double kemna_vorst_block_dlt(const Eigen::MatrixXd _in_m_brownian)
{
    Eigen::VectorXd _v_pd_gao_dlt = pd_gao_dlt(_in_m_brownian);
    Eigen::VectorXd _v_pd_aao_dlt = pd_aao_dlt(_in_m_brownian);
    double _cfs_gao_dlt_mean = cfs_gao_dlt();
    double _pd_gao_dlt_mean = _v_pd_gao_dlt.mean();
    double _pd_aao_dlt_mean = _v_pd_aao_dlt.mean();
    double _var_gao_dlt = (_v_pd_gao_dlt - _pd_gao_dlt_mean).squaredNorm();
    double _c = _var_gao_dlt > 0 ? (_v_pd_aao_dlt - _pd_aao_dlt_mean).dot(
        _v_pd_gao_dlt - _pd_gao_dlt_mean) / _var_gao_dlt : 0;
    return(_pd_aao_dlt_mean - _c * (_pd_gao_dlt_mean - _cfs_gao_dlt_mean));
}
```

A Appendix

A.1 Proof of 6.1.1

$$A^G(0) = e^{-RT} \tilde{\mathbb{E}} \left[\left(\exp \left\{ \frac{1}{T} \int_0^T \ln S(t) dt \right\} - K \right)^+ \right]$$

As $S(t) = S(0) \exp \left\{ \left(R - \frac{1}{2} \sigma^2 \right) t + \sigma \tilde{W}(t) \right\}$, then $\ln S(t) = \ln S(0) + \left(R - \frac{1}{2} \sigma^2 \right) t + \sigma \tilde{W}(t)$ and

$$\begin{aligned} \frac{1}{T} \int_0^T \ln S(t) dt &= \frac{1}{T} \left(T \ln S(0) + \frac{1}{2} \left(R - \frac{1}{2} \sigma^2 \right) T^2 + \sigma \tilde{X}(T) \right) \\ &= \ln S(0) + \frac{1}{2} \left(R - \frac{1}{2} \sigma^2 \right) T + \frac{\sigma}{T} \tilde{X}(T) \end{aligned}$$

where $\tilde{X}$ is a random variable under $\tilde{\mathbb{P}}$ such that $\tilde{\mathbb{E}} \tilde{X}(T) = 0$ and $\text{Var}[\tilde{X}(T)] = \frac{T^3}{3}$.

$$\begin{aligned} \tilde{\mathbb{E}} \left[\left(\exp \left\{ \frac{1}{T} \int_0^T \ln S(t) dt \right\} - K \right)^+ \right] &= \tilde{\mathbb{E}} \left[\left(S(0) \exp \left\{ \frac{1}{2} \left(R - \frac{1}{2} \sigma^2 \right) T + \frac{\sigma}{T} \tilde{X}(T) \right\} - K \right)^+ \right] \\ &= \tilde{\mathbb{E}} \left[\left(S(0) \exp \left\{ \frac{1}{2} \left(R - \frac{1}{2} \sigma^2 \right) T + \frac{\sigma}{T} \tilde{X}(T) \right\} - K \right) \mathbb{I}_{\{S(0) \exp \left\{ \frac{1}{2} \left(R - \frac{1}{2} \sigma^2 \right) T + \frac{\sigma}{T} \tilde{X}(T) \right\} > K\}} \right] \\ &= \tilde{\mathbb{E}} \left[S(0) \exp \left\{ \frac{1}{2} \left(R - \frac{1}{2} \sigma^2 \right) T + \frac{\sigma}{T} \tilde{X}(T) \right\} \mathbb{I}_{\{S(0) \exp \left\{ \frac{1}{2} \left(R - \frac{1}{2} \sigma^2 \right) T + \frac{\sigma}{T} \tilde{X}(T) \right\} > K\}} \right] \\ &\quad - K \cdot \tilde{\mathbb{P}} \left\{ \tilde{X}(T) > \frac{T}{\sigma} \ln \left(\frac{K}{S(0)} \right) - \frac{T}{2\sigma} \left(R - \frac{1}{2} \sigma^2 \right) T \right\} = E_1 \end{aligned}$$

Here, take $\lambda_T = \frac{T}{\sigma} \ln \left(\frac{K}{S(0)} \right) - \frac{T^2}{2\sigma} \left(R - \frac{1}{2} \sigma^2 \right)$

$$\begin{aligned} E_1 &= \tilde{\mathbb{E}} \left[S(0) \exp \left\{ \frac{1}{2} \left(R - \frac{1}{2} \sigma^2 \right) T \right\} \exp \left\{ \frac{\sigma}{T} \tilde{X}(T) \right\} \mathbb{I}_{\{\tilde{X}(T) > \lambda_T\}} \right] - K \left(1 - N \left(\lambda_T, 0, \sqrt{\frac{T^3}{3}} \right) \right) \\ &= S(0) \exp \left\{ \frac{1}{2} \left(R - \frac{1}{2} \sigma^2 \right) T \right\} \frac{1}{\sqrt{\frac{2}{3} \pi T^3}} \int_{\lambda_T}^{\infty} e^{\frac{\sigma}{T} x} e^{-\frac{3x^2}{2T^3}} dx - K \left(1 - N \left(\lambda_T, 0, \sqrt{\frac{T^3}{3}} \right) \right) \\ &= S(0) e^{\frac{1}{2} RT - \frac{1}{12} \sigma^2 T} \left(1 - N \left(\lambda_T, \frac{\sigma T^2}{3}, \sqrt{\frac{T^3}{3}} \right) \right) - K \left(1 - N \left(\lambda_T, 0, \sqrt{\frac{T^3}{3}} \right) \right) \end{aligned}$$

where, defining $\gamma_T = \exp \left\{ -\frac{1}{2} RT - \frac{1}{12} \sigma^2 T \right\}$, and substituting into original equation for $A^{(G)}(0)$ gives:

$$A^G(0) = S(0) \gamma_T \left(1 - N \left(\lambda_T, \frac{\sigma T^2}{3}, \sqrt{\frac{T^3}{3}} \right) \right) - K e^{-RT} \left(1 - N \left(\lambda_T, 0, \sqrt{\frac{T^3}{3}} \right) \right)$$

Which was to be shown. ■

A.2 Linear approximation of Brownian motion (6.2.1)

As seen in 7.1, $w(t_i)$, which is a discrete step of a simulated Brownian motion. On the range $(w(t_i), w(t_{i+1}))$, $\widetilde{W}(t)$ is a linear function:

$$\widetilde{W}_{(w(t_i), w(t_{i+1}))}(t) = \frac{w(t_{i+1}) - w(t_i)}{t_{i+1} - t_i}(t - t_i) + w(t_i)$$

Substituting into the formula for $\bar{S}(T)$:

$$\begin{aligned} \bar{S}(T) &= \frac{1}{T} \int_0^T S(0) \exp \left\{ t \left(R - \frac{1}{2} \sigma^2 \right) + \sigma \widetilde{W}(t) \right\} dt \\ &= \frac{S(0)}{T} \sum_{i=0}^{i_T} \int_{t_i}^{t_{i+1}} \exp \left\{ t \left(R - \frac{1}{2} \sigma^2 \right) + \sigma \widetilde{W}(t) \right\} dt \\ &= \frac{S(0)}{T} \sum_{i=0}^{i_T} \int_{t_i}^{t_{i+1}} \exp \left\{ t \left(R - \frac{1}{2} \sigma^2 \right) + \sigma \widetilde{W}_{(w(t_i), w(t_{i+1}))}(t) \right\} dt \\ &= \frac{S(0)}{T} \sum_{i=0}^{i_T} \int_{t_i}^{t_{i+1}} \exp \left\{ t \left(R - \frac{1}{2} \sigma^2 + \frac{w(t_{i+1}) - w(t_i)}{t_{i+1} - t_i} \cdot \sigma \right) - \sigma \left(\frac{w(t_{i+1}) - w(t_i)}{t_{i+1} - t_i} \cdot t_i - w(t_i) \right) \right\} dt \end{aligned}$$

Which gives the equation in form as used in 7.1:

$$\bar{S}(T) = \frac{S(0)}{T} \sum_{i=0}^{i_T} \left(\frac{1}{\alpha_i} e^{\beta_i} (e^{\alpha_i t_{i+1}} - e^{\alpha_i t_i}) \right)$$

A.3 Proof of 6.2.2

$$\bar{S}(T) = \exp \left\{ \frac{1}{T} \int_0^T \ln S(t) dt \right\} = S(0) \exp \left\{ \frac{1}{2} \left(R - \frac{1}{2} \sigma^2 \right) T^2 + \frac{\sigma}{T} \tilde{X}(T) \right\}$$

Hence, taking $\frac{\partial \bar{S}(T)}{\partial S(0)}$, get the obvious result:

$$\frac{\partial \bar{S}(T)}{\partial S(0)} = \exp \left\{ \frac{1}{2} \left(R - \frac{1}{2} \sigma^2 \right) T^2 + \frac{\sigma}{T} \tilde{X}(T) \right\} = \frac{\bar{S}(T)}{S(0)}$$

Which was to be shown. ■